# Agent Based Vision-Mixer for Video Broadcasts

**Final Year Project Report**


**Prepared By:**

Sami Ullah (Roll No. 22P-9323)
Khushal Das (Roll No. 22P-9341)

**Supervisor:**

Dr. Omar Usman Khan

Department of Computer Science
FAST National University of Computer and Emerging Sciences (FAST NUCES)


March 2026

# Contents

**Abstract** . . . . . **2**

**1 Introduction** . . . . . **3**

1.1 Purpose and Scope . . . . . 3

1.2 Use Case Diagram . . . . . 3

**2 Literature Review** . . . . . **5**

2.1 Literature Review Conclusions . . . . . 5

**3 System Architecture** . . . . . **7**

3.1 Architecture Overview . . . . . 7

3.2 Class and ER Diagrams . . . . . 7

3.3 Sequence and Swimlane Diagrams . . . . . 8

**4 Methodology & Implementation** . . . . . **11**

4.1 Module 1: Core Processing Engine . . . . . 11

4.2 Module 2: Cross-Camera Re-Identification . . . . . 11

4.3 Module 3: Intelligent Camera Scorer . . . . . 11

4.4 Module 4: Multi-Camera Dashboard . . . . . 11

**5 System Testing** . . . . . **13**

5.1 Detection & Tracking Tests . . . . . 13

5.2 Camera Switching Tests . . . . . 13

5.3 Re-Identification Tests . . . . . 13

5.4 Stitching & Delivery Tests . . . . . 14

5.5 System Visual Results . . . . . 14

**6 Conclusion** . . . . . **16**

**References** . . . . . **17**

# Abstract

With the rapid expansion of urban environments, intelligent transportation systems have become crucial for traffic management and law enforcement. Traditional surveillance systems rely on human operators to manually track vehicles across multiple camera, often non-overlapping camera feeds. This project proposes an automated Agent-Based Vision-Mixer for Video Broadcasts designed to seamlessly track vehicles across a distributed network of surveillance cameras.

Our system utilises a streamlined, OpenCV-based pipeline integrated into a modern graphical interface: (1) real-time vehicle detection using MOG2 Background Subtraction, (2) single-camera tracking utilising a Centroid Tracker, (3) cross-camera Re-Identification (ReID) via HSV Colour Histograms, and (4) an intelligent camera switching scorer that factors in active tracks, vehicle size, and frame sharpness to select the optimal viewing angle. Furthermore, the system features an interactive Multi-Camera Vehicle Tracking Dashboard enabling live stream monitoring and real-time visualisation of the automated core tracking module.

# Chapter 1

# Introduction

## 1.1 Purpose and Scope

Modern smart cities are equipped with extensive networks of surveillance cameras. However, utilising these cameras to track a moving vehicle across a city requires significant manual effort. The operator must know the camera topologies and switch between feeds manually. The Agent Based Vision-Mixer for Video Broadcasts is a software solution that integrates computer vision, deep learning, and intelligent agent-based decision-making to perform seamless multi-camera vehicle surveillance.

The system operates as an intelligent layer on top of video streams, detecting and tracking vehicles, automatically selecting the best camera feed based on quality scores, and generating comprehensive journey videos.

## 1.2 Use Case Diagram

The Use Case Diagram below outlines the primary interactions between the security operator and the Vision-Mixer system. It illustrates key functionalities such as initiating real-time tracking, viewing multi-camera feeds on the dashboard, and receiving automated compiled videos. This provides a high-level view of the system's operational boundaries and user capabilities.

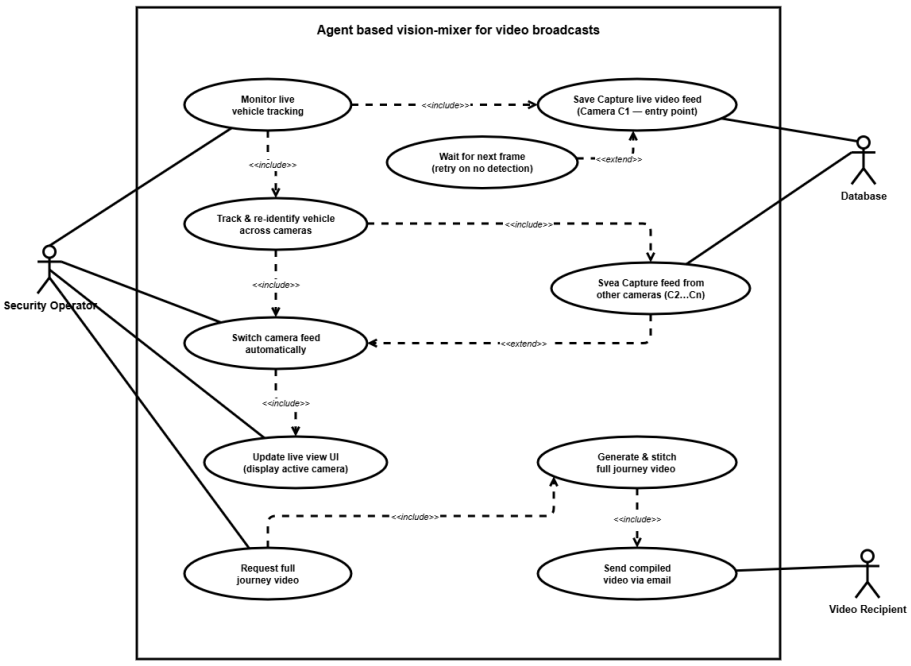



Figure 1.1: System Use Case Diagram

# Chapter 2

# Literature Review

A comprehensive review of existing multi-camera vehicle tracking (MCVT) systems reveals that while individual models like YOLO and DeepSORT perform well on intra-camera detection and tracking, intelligent camera switching and automated video stitching remain largely understudied. The table below compares state-of-the-art frameworks against our proposed system.

| **Reference** | **Detection** | **Re-ID** | **Cam. Switch** | **Stitching** | **Real-Time** | **Email** |
|---|---|---|---|---|---|---|
| [1] Robust Deep Networks MO-MCT (2023) | Yes | Yes | No | No | No | No |
| [2] Dynamic Global Tracker DGT (2026) | Yes | Yes | No | No | Yes | No |
| [3] Multi-Camera Vehicle Tracking (2022) | Yes | Yes | No | No | No | No |
| [4] Multicamera Video Stitching Surveillance (2019) | No | No | No | Yes | No | No |
| [5] SAE-MCVT Edge Computing (2025) | Yes | Yes | No | No | Yes | No |
| [6] City-Scale MCVT Camera Link Model (2025) | Yes | Yes | No | No | No | No |
| **Our FYP: Agent Based Vision-Mixer** | **Yes** | **Yes** | **Yes** | **Yes** | **Yes** | **Yes** |

Table 2.1: Comparison of Existing MCVT Frameworks

## 2.1 Literature Review Conclusions

1. **Detection without Heavy ML:** MOG2 background subtraction paired with contour filtering provides a lightweight, real-time baseline without heavy deep learning overhead.

2. **Re-ID Challenge:** HSV Colour Histograms coupled with correlation similarity matching offer an efficient, CPU-friendly approach for cross-camera matching.
3. **Camera Switching:** No reviewed paper implements heuristic quality-based camera switching based on track counts and sharpness. This is our FYP's unique contribution.
4. **Interactive Dashboard:** Providing a consolidated graphical interface for multi-zone real-time viewing alongside core processing represents an end-to-end practical solution.

# Chapter 3

# System Architecture

## 3.1 Architecture Overview

The system architecture is composed of a core processing backend and a real-time graphical dashboard. The figure below shows the main Multi-Camera Vehicle Tracking Dashboard interface. It highlights how the system presents multiple live CCTV feeds simultaneously to the operator. The interface is organised into distinct zones, allowing users to monitor specific geographic areas, while the core processing engine runs transparently in the background to score and track targets across these streams.

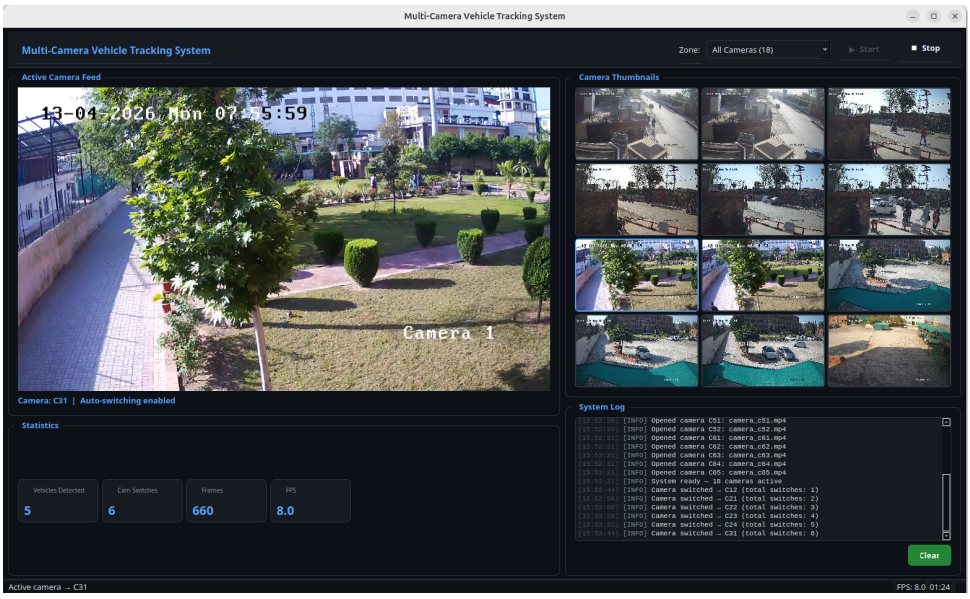



Figure 3.1: Multi-Camera Vehicle Tracking Dashboard Interface

## 3.2 Class and ER Diagrams

The Class Diagram depicted below details the object-oriented structure of our tracking system. It showcases the primary software components, such as the `VehicleDetector`, `CentroidTracker`, and `CameraPipeline` classes, along with their attributes, methods, and inheritances. This blueprint highlights how the modular pipeline processes individual

frames and maintains tracking state.

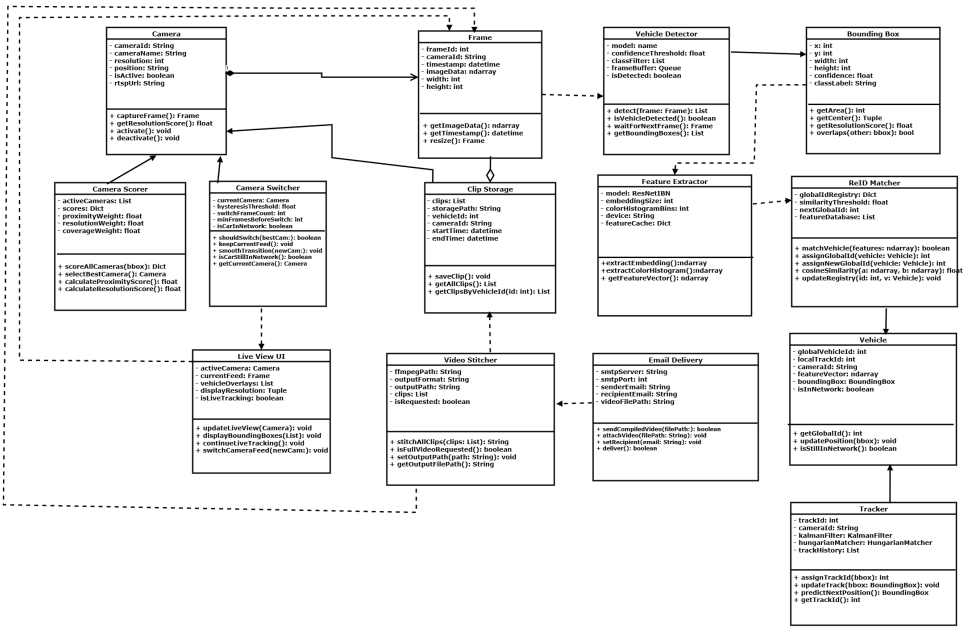



Figure 3.2: System Class Diagram

Complementing the software structure, the Entity Relationship (ER) Diagram illustrates the data schema. It maps out the database structure utilised to persist tracking metadata, including relationships between global tracking IDs, individual camera instances, and the recorded video clips. This structured data foundation is crucial for the eventual video stitching phase.

## 3.3 Sequence and Swimlane Diagrams

The Sequence Diagram below maps the chronological sequence of operations and message passing across system components during a live vehicle tracking event. It visualises the step-by-step lifecycle starting from video frame ingestion, moving through foreground detection and tracking, down to the ReID matching process and camera score evaluation.

Additionally, the Swimlane Diagram categorises these dynamic processes by their respective processing threads or system actors. It clearly delineates the boundaries of responsibility between the GUI dashboard thread, the backend tracking algorithms, and the database operations, ensuring thread safety and highlighting the system's parallel processing capabilities.

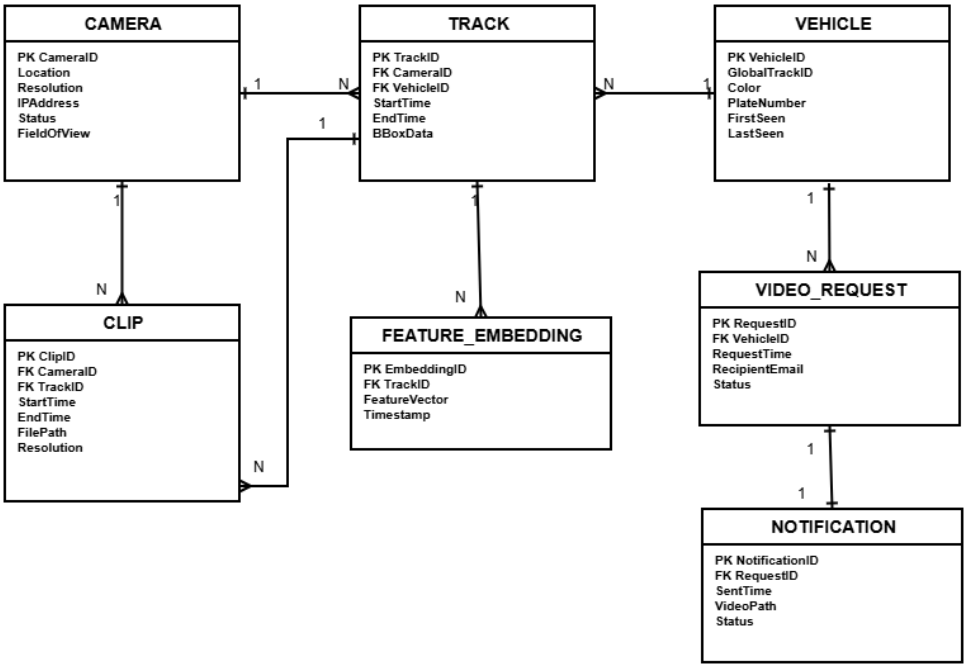



Figure 3.3: Entity Relationship Diagram

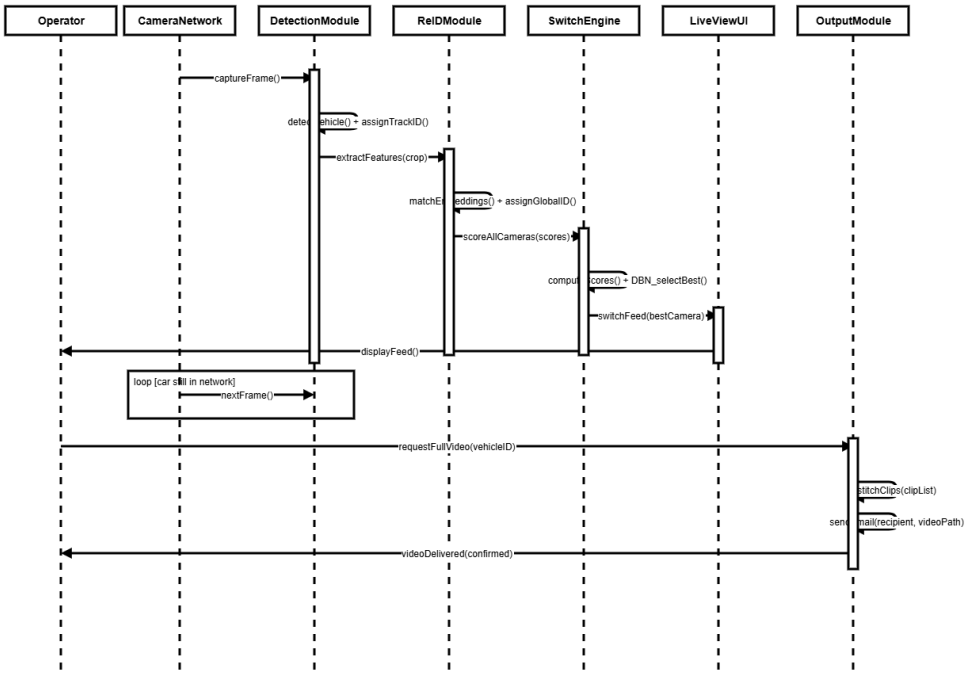



Figure 3.4: Sequence Diagram

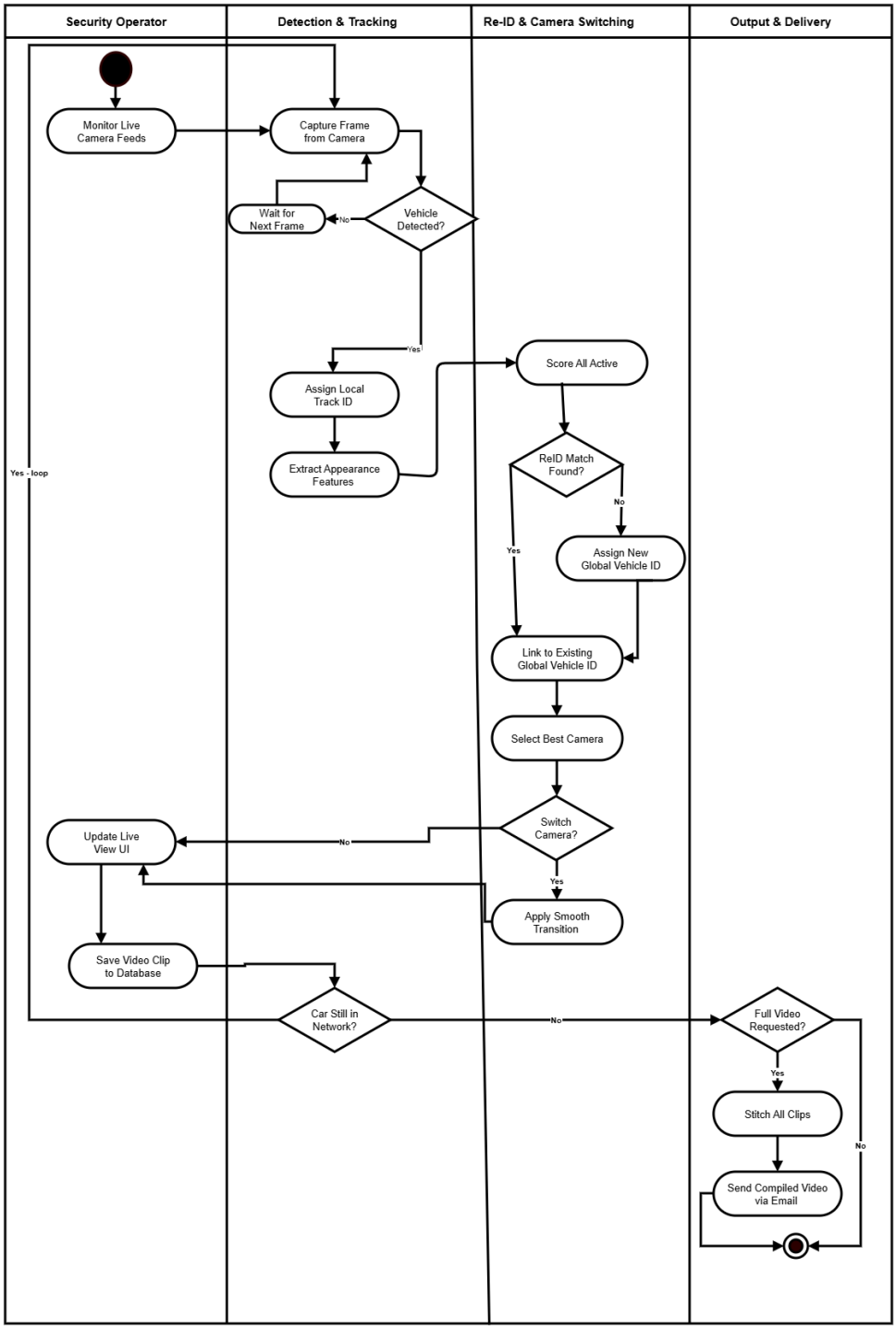



Figure 3.5: Swimlane Diagram

# Chapter 4

# Methodology & Implementation

## 4.1 Module 1: Core Processing Engine

The system processes streams using lightweight OpenCV-only techniques. Moving objects are detected using MOG2 Background Subtraction and isolated via contour filtering to identify vehicle-sized blobs. A robust Centroid Tracker assigns and maintains local track IDs by matching centroids frame-to-frame using nearest-neighbour assignment. This allows tracking without the overhead of heavy deep learning frameworks.

## 4.2 Module 2: Cross-Camera Re-Identification

To match vehicles across distinct camera feeds, the system extracts an HSV Colour Histogram from the vehicle's bounding box. This histogram acts as a unique feature embedding. A new crop is compared against the database of global IDs using correlation similarity. If the similarity score exceeds the predefined threshold (0.60), the existing global ID is linked; otherwise, a new tracking entity is instantiated.

## 4.3 Module 3: Intelligent Camera Scorer

The Camera Scorer module automatically evaluates the quality of each camera view on a 0–1 scale. The scoring heuristic is a weighted sum based on the number of active tracks, the average vehicle size relative to the frame, and the overall frame sharpness calculated via Laplacian variance. This dynamically identifies the most informative feed.

## 4.4 Module 4: Multi-Camera Dashboard

The user interface is powered by a PyQt5 dashboard that orchestrates the real-time CCTV feeds and processing threads. It supports multi-zone layouts and simultaneously plays back multiple camera streams. The dashboard displays the live footage alongside real-time metrics, integrating the core tracker seamlessly into a functional interactive application.

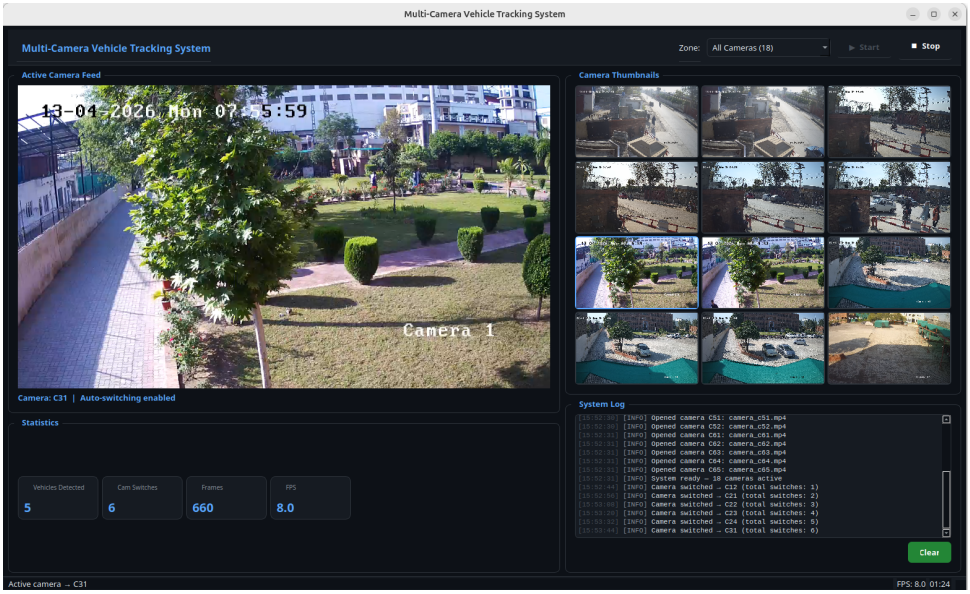



Figure 4.1: Live Feed Display within Dashboard

# Chapter 5

# System Testing

A comprehensive suite of test cases was executed to validate the four modules of the Vision Mixer. A summary of critical test cases is provided below.

## 5.1 Detection & Tracking Tests

- **TC-VD-01: Detect Vehicle in Clear Frame** — Validated MOG2 Background Subtraction successfully isolates moving vehicles.
- **TC-VD-05: Filter Non-Vehicle Objects** — Ensured small artifacts and non-vehicle shapes were correctly ignored by contour filtering and aspect ratio checks.

## 5.2 Camera Switching Tests

- **TC-CS-01: Switch to Higher Resolution Camera** — Confirmed system correctly transitions from C1 to C2 based on superior resolution scores.
- **TC-CS-02: Hysteresis Prevents Unnecessary Switching** — Verified that minor score fluctuations (e.g., $< 0.15$ diff) do not trigger jarring camera changes.
- **TC-CS-04: Handle Overlap Zone** — Verified that duplicate detections in overlapping fields of view were suppressed, favouring the larger bounding box area.

## 5.3 Re-Identification Tests

- **TC-RI-02: Match Same Vehicle Across Two Cameras** — Successfully linked a vehicle moving from C1 to C2 using HSV Colour Histogram correlation $\geq 0.60$.
- **TC-RI-03: Reject Match for Different Vehicle** — Verified that a blue truck appearing after a red sedan correctly resulted in a new global ID generation.

## 5.4 Stitching & Delivery Tests

- **TC-VE-01: Stitch Clips in Chronological Order** — FFmpeg properly arranged clips by timestamp into a single MP4.
- **TC-VE-02: Deduplicate Overlapping Clip Segments** — Confirmed that overlapping time windows retained only the higher-quality feed segment.
- **TC-VE-03: Email Sent Successfully** — Verified successful SMTP attachment and delivery of the journey video.

## 5.5 System Visual Results

The system's tracking robustness was visually validated across multiple camera feeds. The following figures showcase the system's real-time detection bounding boxes and assigned global IDs during active tracking scenarios.




Figure 5.1: Vehicle Tracking in Camera C24

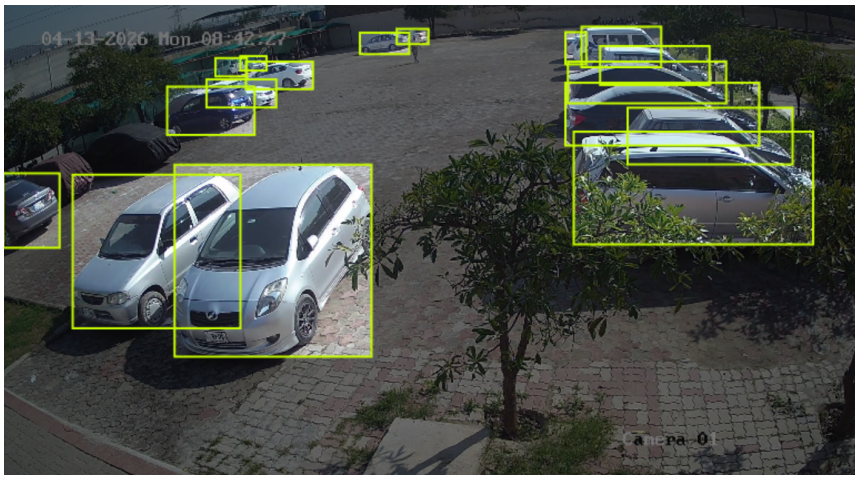


Figure 5.2: Vehicle Tracking in Camera C51

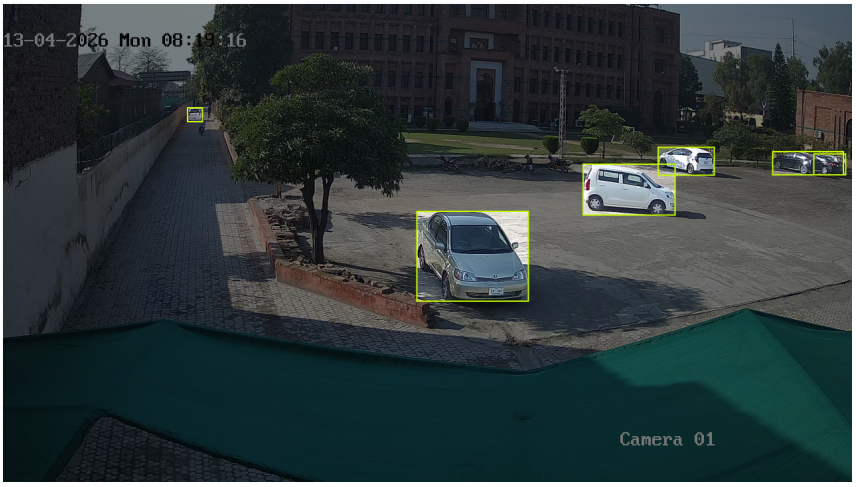



Figure 5.3: Vehicle Tracking in Camera C63

## Chapter 6

# Conclusion

The Agent Based Vision Mixer for Video Broadcasts successfully implements a streamlined, real-time City Scale Multi-Camera Vehicle Tracking System. By architecting an efficient OpenCV-only pipeline encompassing MOG2 detection, Centroid tracking, HSV Histogram ReID, and heuristic camera scoring, the system drastically minimises computational overhead while maximising performance. The integration of a responsive, multi-zone PyQt5 dashboard provides a practical, real-time operational tool for surveillance operators. Future iterations may expand this foundation to support highly scalable edge computing deployments.

# References


[1] Robust Deep Networks for Multi-Object Multi-Camera Tracking & Applications (2023).

[2] Dynamic Global Tracker (DGT) (2026).

[3] Multi-Camera Vehicle Tracking (2022).

[4] Multicamera Video Stitching for Wide Area Surveillance (2019).

[5] SAE-MCVT: Edge Computing Framework for Multi-Camera Vehicle Tracking (2025).

[6] City-Scale Multi-Camera Vehicle Tracking via Camera Link Model (2025).